

119.2. La fonction sauvegarde() — relire un nombre gardé

#include <sauvegarde> : `sauvegarde(numero)` relit une valeur écrite par `sauver()`, même d'une partie précédente.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `sauvegarde(numero)` **relit** la case de la cartouche : ce qu'un `sauver()` y a écrit, aujourd'hui ou la veille.

La ligne à écrire : `#include <sauvegarde>`.

Son argument : le numéro de la case.

Ce qu'elle coûte : une vingtaine d'octets.

Attention : une cartouche **neuve** ne contient rien de sûr. On y range d'abord une « marque » : si elle n'y est pas, on remet tout à zéro.

On retrouve sauvegarde dans 3 leçons

[120](#), [135](#), [138](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **494 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	52 o	← la console, à l'allumage — la mise en route de la console
main()	140 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireNombre — attend de pouvoir écrire une case
EcrireNombre	33 o	← nombre() — écrit un nombre en chiffres
CopierTuiles	31 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	160 o	← CopierTuiles — 10 lettres sur 43

Lettres gravées : 0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

sauver()	× 3	73 o sur place, dans main()
sauvegarde()	× 2	41 o sur place, dans main()
nombre()	× 1	10 o sur place, dans main() + EcrireNombre (33 o)
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <sauvegarde> : relire un nombre gardé ----  
// Le nombre de fois où la console a été allumée : gardé dans la case 1.  
  
#include <sauvegarde>    // sauvegarde() : relire la cartouche  
#include <sauver>  
#include <nombre>
```

2. UNE VARIABLE — ALLUMAGES

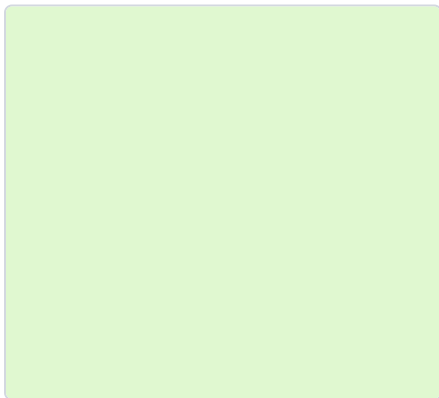
```
uint8_t allumages = 0;
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    if (sauvegarde(0) != 42) {           // pas de marque : une cartouche neuve  
        sauver(0, 42);                  // on pose la marque  
        sauver(1, 0);                   // et le compte à zéro  
    }  
    allumages = sauvegarde(1) + 1;       // relire, et compter cet allumage  
    sauver(1, allumages);  
    nombre(8, 8, allumages);  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

001 au premier allumage — et un de plus à chaque fois qu'on rallume.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ premier allumage : 1
- ✓ la marque et le compte sont dans la cartouche
- ✓ 001 s'affiche

Ensuite : **120. Le record, gardé dans la cartouche** — chapitre 20 : Les états du jeu : titre, partie, fin. · gameboy3 — écrire une cartouche Game Boy en C++